

Departement Informatik
Hochschule Luzern (Schweiz)

Faire Wohnungsvergabe Tool

WEB PROGRAMMING LAB PROJEKT

vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

von

Noah Emmenegger

von

Zug (Schweiz)

Web Programming Lab Projekt vorgelegt am Departement Informatik der Hochschule Luzern (Schweiz)

Titel: Faire Wohnungsvergabe Tool

Student: Noah Emmenegger

Studiengang: BSc Informatik

Jahr: 2026

Betreuungsperson: Dozent/in

Auftraggeberin / Auftraggeber: Web Programming Lab

Kurzfassung

[Kurzfassung des Projekts]

Inhaltsverzeichnis

1	Einführung und Ziele	10
1.1	Aufgabenstellung	10
1.1.1	Fachlicher Umfang	10
1.1.2	Abgrenzung	12
1.2	Qualitätsziele	12
1.3	Stakeholder	12
2	Randbedingungen	14
2.1	Technische Randbedingungen	14
2.2	Organisatorische Randbedingungen	14
2.3	Konventionen	14
3	Kontextabgrenzung	15
3.1	Fachlicher Kontext	15
3.2	Technischer Kontext	15
4	Lösungsstrategie	17
4.1	Technologie-Stack	17
4.2	Architekturansatz	18
4.3	Entwicklungsvorgehen	19
5	Bausteinsicht	20
5.1	Ebene 1: Gesamtsystem	20
5.2	Ebene 2: Next.js-Anwendung	20
5.3	Ebene 2: Persistenz	21
6	Laufzeitsicht	23
6.1	Bewerbung absenden	23
6.2	Bewertete Bewerbungen laden	23
6.3	Datenbank beim gemeinsamen Start noch nicht bereit	23
6.4	Aktueller API-Datenfluss	24
7	Verteilungssicht	25
7.1	Container und Laufzeit	25
7.2	Reproduzierbarer Start	26
7.3	CI/CD und automatisches Deployment mit Dokploy	26
8	Querschnittliche Konzepte	28
8.1	Domänenmodell und Persistenz	28
8.2	API- und Schnittstellendesign	28
8.3	Benutzeroberfläche und Responsive Design	28
8.4	Fehlerbehandlung und Logging	28
8.5	Sicherheit	28
8.6	Testkonzept	29
9	Architekturentscheidungen	30
9.1	ADR-001: Next.js statt getrenntem React-Frontend und Express-Backend	30
9.2	ADR-002: PostgreSQL und gemeinsamer Start mit Docker Compose	31
9.3	ADR-003: Drizzle ORM statt Prisma	32
10	Qualitätsanforderungen	33
10.1	Qualitätsbaum	33

10.2	Qualitätsszenarien	33
10.3	Evaluationsergebnisse	33
11	Risiken und technische Schulden	34
12	Glossar	35
13	Fazit und Reflexion	36
13.1	Was ist gut gelaufen?	36
13.2	Wo lagen die Herausforderungen?	36
13.3	Was würde ich beim nächsten Mal anders machen?	36
13.4	Zielerreichung	36
14	Arbeitsjournal	37
15	Anhang	39
15.1	Abgabe und Softwareartefakte	39
15.2	Ergänzende Unterlagen	39

Abkürzungsverzeichnis

Abkürzung	Bedeutung
CMS	Content Management System
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets

Literaturverzeichnis

Abbildungsverzeichnis

Abbildung 1	Fachlicher Kontext der Wohnungsvergabe-Plattform	15
Abbildung 2	Bausteinsicht des Gesamtsystems	20

Tabellenverzeichnis

Tabelle 1	Schichten und Ausführungsorte der Next.js-Anwendung	21
-----------	---	----

Code-Verzeichnis

Code 1	Laufzeitdienste und Datenfluss der Compose-Umgebung	25
Code 2	Vorgesehene CI/CD- und Deployment-Kette	26

1 Einführung und Ziele

Dieses Kapitel beschreibt den fachlichen Anlass, den Umfang, die wichtigsten Qualitätsziele und die beteiligten Personen des Projekts.

1.1 Aufgabenstellung

Immobilien- und Genossenschaftsgesellschaften möchten Wohnungsvergaben möglichst fair und nachvollziehbar durchführen. Heute müssen Ausschreibungen, Bewerbungsformulare und eingehende Bewerbungen häufig mit mehreren voneinander getrennten Hilfsmitteln verwaltet und manuell verglichen werden.

Im Rahmen des Projekts wird deshalb eine webbasierte Plattform für die Vorbereitung und Durchführung von Wohnungsvergaben entwickelt. Mitarbeitende einer Gesellschaft erstellen damit Wohnungsausschreibungen und individuelle Bewerbungsformulare. Vor Beginn einer Ausschreibung definieren sie Bewertungskriterien und deren Gewichtung. Eingehende Bewerbungen werden anschliessend zentral verwaltet, anhand dieser Regeln automatisch bewertet und nach ihrer Übereinstimmung mit den Vergabekriterien sortiert.

Die Plattform unterstützt die Entscheidungsfindung, trifft aber keine endgültige Vergabeentscheidung. Diese bleibt bei der zuständigen Person. Damit die Entscheidung überprüfbar bleibt, zeigt das System für jede Bewerbung transparent auf, wie die Gesamtbewertung zustande gekommen ist.

1.1.1 Fachlicher Umfang

Die Anforderungen sind nach der MoSCoW-Methode priorisiert. Die Must-Have-Stories bilden den geplanten Mindestumfang der ersten Version.

Must Have

- Als Gesellschaft kann ich eine neue Wohnungsausschreibung erstellen.
- Als Gesellschaft kann ich für eine Ausschreibung ein individuelles Bewerbungsformular erstellen.
- Als Gesellschaft kann ich Bewertungskriterien für eine Ausschreibung definieren.
- Als Gesellschaft kann ich festlegen, wie stark die einzelnen Kriterien gewichtet werden.
- Als Gesellschaft kann ich eine Musterbewerbung ausfüllen, um die definierten Kriterien und Gewichtungen zu testen.

- Als Gesellschaft kann ich die Kriterien und Gewichtungen vor Beginn der Ausschreibung festlegen und sperren.
- Als Gesellschaft kann ich das Bewerbungsformular über einen Link teilen.
- Als Gesellschaft kann ich das Bewerbungsformular optional mit einem Zugangsschlüssel schützen.
- Als Bewerber/in kann ich das Bewerbungsformular ausfüllen und absenden.
- Als Gesellschaft kann ich alle eingegangenen Bewerbungen zentral einsehen.
- Als Gesellschaft kann ich die Bewerbungen anhand der definierten Kriterien automatisch bewerten lassen.
- Als Gesellschaft kann ich die Bewerbungen nach ihrer Übereinstimmung mit den Vergabekriterien sortieren.
- Als Gesellschaft kann ich bei einer Bewerbung einsehen, wie sich die einzelnen Kriterien auf die Gesamtbewertung ausgewirkt haben.
- Als Gesellschaft kann ich eine Bewerbung als ausgewählt markieren.
- Als Gesellschaft kann ich Bewerbungen und deren Bewertungen exportieren.

Should Have

- Als Gesellschaft kann ich bestehende Ausschreibungen duplizieren und als Vorlage verwenden.
- Als Gesellschaft kann ich bestehende Bewerbungsformulare wiederverwenden.
- Als Gesellschaft kann ich Mindestanforderungen definieren, die eine Bewerbung erfüllen muss.
- Als Gesellschaft kann ich Bewerbungen filtern und durchsuchen.
- Als Gesellschaft kann ich mehrere Mitarbeitende verwalten.
- Als Gesellschaft kann ich abgeschlossene Ausschreibungen archivieren.
- Als Gesellschaft kann ich einen zusammenfassenden Bericht über eine abgeschlossene Vergabe erstellen.

Could Have

- Als Bewerber/in kann ich den aktuellen Status meiner Bewerbung einsehen.
- Als Gesellschaft kann ich Bewerber/innen direkt über den Status ihrer Bewerbung informieren.
- Als Gesellschaft kann ich Statistiken zu vergangenen Ausschreibungen anzeigen.

- Als Gesellschaft kann ich unterschiedliche Bewertungsmodelle als Vorlagen speichern.
- Als Gesellschaft kann ich einzelne Bewerbungen kommentieren oder intern markieren.

1.1.2 Abgrenzung

Folgende Funktionen sind für den Projektzeitraum ausdrücklich nicht vorgesehen:

- Die Plattform trifft keine endgültige Vergabeentscheidung ohne menschliche Bestätigung.
- Es wird keine künstliche Intelligenz zur Bewertung oder Vorhersage der Eignung von Bewerber/innen eingesetzt.
- Es wird kein ausschreibungsübergreifender Score für Bewerber/innen erstellt.
- Es werden keine externen Datenquellen oder sozialen Netzwerke zur Bewertung verwendet.

1.2 Qualitätsziele

Prio	Qualitätsziel	Begründung	Nachweis
1	Nachvollziehbarkeit	Eine Bewertung muss aus den vorab festgelegten Kriterien, Antworten und Gewichtungen vollständig erklärbar sein.	Berechnungsbeispiele, Unit- und E2E-Tests
2	Benutzbarkeit	Die Kernaufgaben sind auf Desktop und Mobile verständlich und effizient ausführbar.	E2E-Test, manuelle Prüfung, Lighthouse
3	Zuverlässigkeit	Ausschreibungen, Kriterien und Bewerbungen bleiben konsistent und dauerhaft verfügbar.	Integrations- und E2E-Tests
4	Änderbarkeit	Die Anwendung bleibt trotz des begrenzten Projektumfangs nachvollziehbar strukturiert und erweiterbar.	Code-Review, automatisierte Tests

1.3 Stakeholder

Stakeholder	Erwartung	Bezug zum System
Mitarbeitende einer Immobiliengesellschaft	Einfache Verwaltung von Ausschreibungen und Bewerbungen	Hauptbenutzer/innen des geschützten Verwaltungsbereichs

Stakeholder	Erwartung	Bezug zum System
schaft oder Genossenschaft	sowie nachvollziehbare Entscheidungsgrundlagen	
Bewerber/innen	Ein verständliches, responsives und zuverlässig funktionierendes Bewerbungsformular	Füllen ein öffentlich geteiltes und optional geschütztes Formular aus
Verantwortliche der Gesellschaft	Ein fairer, dokumentierbarer Vergabeprozess; die endgültige Entscheidung bleibt in ihrer Verantwortung	Verantworten Kriterien und Vergabeentscheid
Entwickler/in und Betreiber/in	Verständliche Struktur, testbare Fachlogik und reproduzierbarer Betrieb	Entwickelt, wartet und betreibt das System
Dozent/in	Nachvollziehbare Erfüllung der Projektanforderungen	Bewertet Artefakt, Vorgehen und Dokumentation

2 Randbedingungen

Dieses Kapitel enthält Vorgaben, die den Lösungsraum des Projekts einschränken. Gewählte Technologien und Architekturansätze werden dagegen in Kapitel 4 beschrieben.

2.1 Technische Randbedingungen

- Die Anwendung muss Daten persistent in einer Datenbank speichern.
- Die Benutzeroberfläche muss auf Desktop, Tablet und Mobile responsiv nutzbar sein.
- Automatisierte Unit-, Integrations- und/oder E2E-Tests müssen passend zu den Projektrisiken eingesetzt werden.
- Ein Produktions-Build muss über eine öffentliche URL oder mit `docker compose up` reproduzierbar startbar sein.
- Der durchschnittliche Lighthouse-Score muss für Mobile und Desktop mindestens 90 betragen.
- Sämtliche Artefakte werden in einem Git-Repository abgegeben.

2.2 Organisatorische Randbedingungen

- Das Projekt wird als Einzelarbeit mit einem vorgesehenen persönlichen Arbeitsaufwand von rund 60 Stunden durchgeführt.
- Die Entwicklung erfolgt strukturiert und systematisch.
- Wesentliche Architekturentscheidungen müssen nachvollziehbar begründet werden.
- Die Dokumentation richtet sich an Software Engineers und bleibt kurz, präzise und diagrammorientiert.

2.3 Konventionen

- Quellcode und Dokumentation werden gemeinsam mit Git versioniert.
- Die Architekturdokumentation orientiert sich an der Kapitelstruktur von arc42.
- Fachbegriffe werden im gesamten System und in der Dokumentation einheitlich verwendet; projektspezifische Begriffe werden im Glossar erläutert.
- Eine User Story gilt erst als abgeschlossen, wenn sie umgesetzt, angemessen getestet und in der Dokumentation beziehungsweise im Arbeitsjournal berücksichtigt ist.

3 Kontextabgrenzung

Dieses Kapitel grenzt die Wohnungsvergabe-Plattform von ihren Benutzergruppen und der technischen Umgebung ab. In der ersten Version bestehen keine Schnittstellen zu externen Fachsystemen oder Datenquellen.

3.1 Fachlicher Kontext

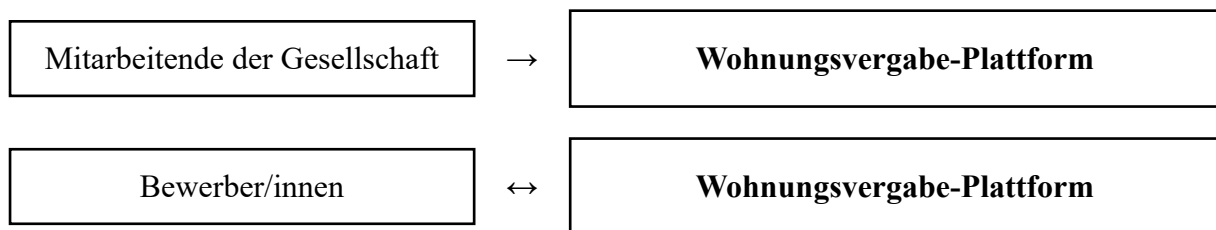


Abbildung 1: Fachlicher Kontext der Wohnungsvergabe-Plattform

Akteur	Schnittstelle	Ausgetauschte Informationen
Mitarbeitende der Gesellschaft	Geschützte Weboberfläche	Ausschreibungen, Formulare, Kriterien, Gewichtungen, Musterbewerbungen, Bewerbungen, Auswahlstatus und Exporte
Bewerber/innen	Öffentlich geteiltes Webformular, optional mit Zugangsschlüssel	Formularfelder, Bewerbungsdaten und - je nach umgesetztem Umfang - Bewerbungsstatus

Die Gesellschaft legt die Vergaberegeln fest und erhält die aufbereiteten Bewertungen. Bewerber/innen übermitteln ausschliesslich die im jeweiligen Formular verlangten Angaben. Die endgültige Vergabeentscheidung entsteht ausserhalb des Systems durch die verantwortliche Person; im System wird sie lediglich dokumentiert.

3.2 Technischer Kontext

Die Benutzer/innen greifen mit einem aktuellen Webbrowser über HTTPS auf die Anwendung zu. Der Next.js App Router ordnet die eingehenden Anfragen den Seiten und serverseitigen Endpunkten zu. Benutzeroberfläche, serverseitige Validierung und Fachlogik werden als eine Next.js-Anwendung betrieben; es existiert kein separater Express-Server. Die Next.js-Anwendung persistiert die Daten über ein ORM in PostgreSQL. Datenexporte werden als Datei über die Weboberfläche bereitgestellt.

Kommunikations-partner	Kanal/Protokoll	Bemerkung
Browser der Mitarbeitenden	HTTPS; HTML, CSS und JavaScript	Zugriff auf den authentisierten Verwaltungsbereich
Browser der Bewerber/innen	HTTPS; HTML, CSS und JavaScript	Zugriff über einen Freigabelink; optional durch einen Zugangsschlüssel geschützt
Next.js-Anwendung	Interne Modulaufrufe	Routing, Rendering, serverseitige Endpunkte, Fachlogik und Datenzugriff in einem auslieferbaren Artefakt
PostgreSQL-Datenbank	Datenbankprotokoll über ORM	Persistenz für Ausschreibungen, Formulare, Kriterien, Bewerbungen und Bewertungen
Dateisystem des Browsers	HTTPS-Download	Export der Bewerbungen und ihrer Bewertungen

Externe Datenquellen, soziale Netzwerke und KI-Dienste liegen ausserhalb der Systemgrenze und werden nicht angebunden.

4 Lösungsstrategie

Dieses Kapitel fasst die grundlegenden Entscheidungen zusammen, mit denen die fachlichen Ziele und Qualitätsziele erreicht werden. Detaillierte und später revidierbare Entscheidungen werden in Kapitel 9 als Architecture Decision Records festgehalten.

4.1 Technologie-Stack

Bereich	Technologie	Begründung und Zielbeitrag
Web-Framework	Next.js mit App Router und TypeScript	Routing, Frontend und Backend können in einer Anwendung entwickelt und als gemeinsames Artefakt ausgeliefert werden. TypeScript unterstützt die konsistente Modellierung der fachlichen Daten.
Benutzeroberfläche	React und Tailwind CSS	Komponentenbasierte Umsetzung der Verwaltungsoberfläche und der dynamischen, responsiven Bewerbungsformulare.
Backend	Next.js Route Handlers und/oder Server Actions auf Node.js	Serverseitige Validierung, Fachlogik und Datenzugriff bleiben innerhalb des Monolithen gebündelt; ein separater Express-Service entfällt.
Datenbank	PostgreSQL	Relationale und transaktionale Speicherung der miteinander verknüpften Ausschreibungen, Kriterien und Bewerbungen.
Datenzugriff	Drizzle ORM	Typsicherer, SQL-naher Datenzugriff; Tabellen und Migrationen bleiben direkt bei den TypeScript-Modellen.
Authentisierung	Einfache Benutzeranmeldung für Gesellschaften	Schützt den Verwaltungsbereich; öffentlich geteilte Formulare können zusätzlich einen Zugangsschlüssel verlangen.
Tests	Unit-, Integrations- und E2E-Tests; Werkzeuge noch offen	Die deterministische Bewertungslogik, Datenintegrität und zentralen Benutzerabläufe werden automatisiert abgesichert.

Bereich	Technologie	Begründung und Zielbeitrag
Betrieb	Docker Compose und Dokploy als Deployment-Ziel	Compose startet Web-Anwendung, Migration und PostgreSQL in definierter Reihenfolge. Ein Volume persistiert die Datenbankdaten; Dokploy ist für das automatische Deployment vorgesehen.
Versionsverwaltung	Git und GitHub	Nachvollziehbare Entwicklung und gemeinsame Ablage aller Artefakte.
CI/CD	GitHub Actions und Dokploy (geplant)	Typecheck, Lint und Build sollen vor einem automatischen Deploy-Trigger erfolgreich durchlaufen.

4.2 Architekturansatz

Die Anwendung wird als modular strukturierter Monolith umgesetzt. Der Next.js App Router, React-Komponenten, serverseitige Endpunkte und Fachlogik befinden sich in derselben Next.js-Anwendung und werden gemeinsam ausgeliefert. Ein eigenständiges Express-Backend und die damit verbundene zweite Projekt-, Schnittstellen- und Deployment-Konfiguration entfallen. Innerhalb des Monolithen werden die fachlichen Verantwortlichkeiten - insbesondere Ausschreibungen, Formulare, Kriterien und Bewertung, Bewerbungen, Benutzerverwaltung und Export - klar voneinander getrennt. Dadurch bleibt der Betrieb einfach, ohne auf eine nachvollziehbare und erweiterbare Struktur zu verzichten.

Next.js Server Components sind der Standard für Layouts und nicht interaktive Darstellung. Navigation, Footer und datenlesende Seiten werden serverseitig gerendert. Client Components bilden nur die interaktiven Teilbäume, die Browserzustand, Ereignisbehandlung oder unmittelbare Aktualisierungen benötigen, beispielsweise dynamische Formularfelder, Filter und Sortierungen. Damit wird nur so viel JavaScript wie nötig an den Browser ausgeliefert, ohne auf eine reaktive Bedienung zu verzichten.

Die Bewertung einer Bewerbung erfolgt regelbasiert und deterministisch. Antworten werden mit den vor Beginn einer Ausschreibung festgelegten Kriterien verglichen und anhand der gespeicherten Gewichtungen zu einer Gesamtbewertung zusammengeführt. Kriterien und Gewichtungen werden mit dem Start der Ausschreibung gesperrt. Neben dem Gesamtwert spei-

chert beziehungsweise liefert die Anwendung die Beiträge der einzelnen Kriterien, damit das Ranking jederzeit erklärt und mit Berechnungsbeispielen getestet werden kann.

PostgreSQL stellt die konsistente Persistenz der fachlichen Daten sicher. Schreiboperationen werden serverseitig validiert. Docker Compose beschreibt die Laufzeitservices web, migrate und db, ihr internes Netzwerk, die Datenbankkonfiguration und ein persistentes Volume. Der geplante CI/CD-Prozess prüft Änderungen automatisiert und stösst anschliessend ein Deployment auf Dokploy an.

Im aktuellen Stand sind Anmeldung, Gesellschaften, Wohnungen, Ausschreibungen, öffentliche Bewerbungsformulare und CSV-Export umgesetzt. Server Components laden geschützte beziehungsweise öffentliche Daten; Server Actions prüfen und speichern Änderungen über Drizzle in PostgreSQL. GET /api/applications bleibt als einfacher technischer Leseendpunkt vorhanden.

4.3 Entwicklungsvorgehen

Die Entwicklung erfolgt iterativ entlang der priorisierten User Stories. Zuerst wird ein durchgängiger Kernablauf von der Ausschreibung bis zur bewerteten Bewerbung umgesetzt; weitere Must-, Should- und Could-Have-Stories werden anschliessend in dieser Reihenfolge ergänzt. Jede Iteration umfasst Implementierung, angemessene automatisierte Tests, eine kurze Überprüfung der Qualitätsziele und einen Eintrag im Arbeitsjournal. Technisch riskante oder noch offene Entscheide werden früh mit kleinen Versuchen geprüft und als ADR dokumentiert.

5 Bausteinsicht

5.1 Ebene 1: Gesamtsystem

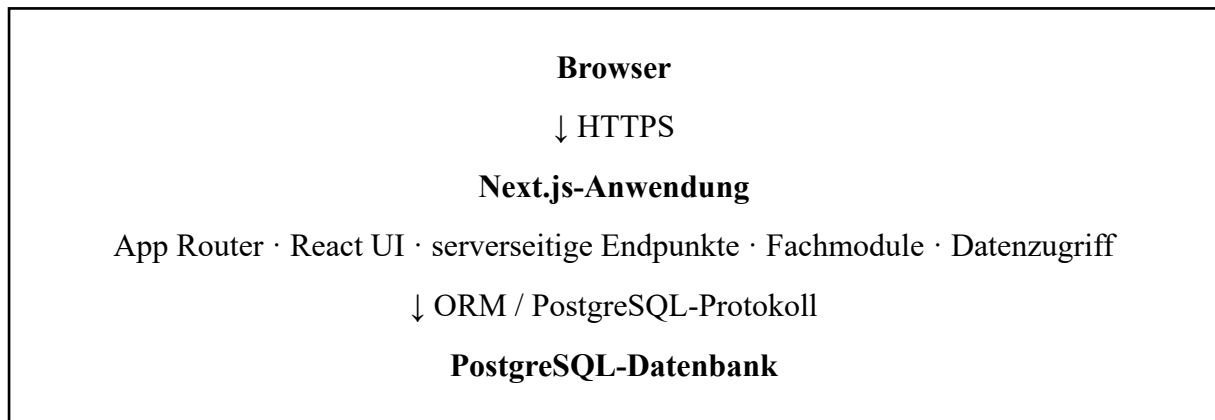


Abbildung 2: Bausteinsicht des Gesamtsystems

Baustein	Verantwortung	Schnittstellen
App Router und UI	Ordnet URLs zu, rendert Seiten und verarbeitet Benutzerinteraktionen.	HTTPS sowie interne Aufrufe
Serverseitige Endpunkte	Authentisierung, Validierung und Orchestrierung der Anwendungsfälle mit Route Handlers und/oder Server Actions.	HTTP oder interne Server-Aufrufe
Fachmodule	Regeln für Ausschreibungen, Formulare, Kriterien, Bewerbungen, Bewertungen und Export.	Typsichere Modulgrenzen
Datenzugriff	Kapselt Abfragen, Transaktionen und Schema-Migrationen.	ORM
PostgreSQL	Persistente, relationale und transaktionale Datenhaltung.	PostgreSQL-Protokoll

5.2 Ebene 2: Next.js-Anwendung

Die Anwendung ist in Schichten mit gerichteten Abhängigkeiten gegliedert. Der App Router und die Komponenten bilden die Darstellung. Server Actions und Route Handlers nehmen Eingaben entgegen und koordinieren Anwendungsfälle. Datenbankfreie Funktionen in `lib/` enthalten Typen, Gewichtungs- und Bewertungsregeln. Die Abfragen in `db/queries/` kapseln

Drizzle und PostgreSQL. Nur serverseitige Einstiegspunkte importieren den Datenzugriff; die Oberfläche erhält Daten als Props oder als Rückgabe einer Server Action.

Ausführungsort	Schicht und Datenfluss
Browser	components/ mit "use client": Formulare, Dialoge, Dashboard-Zustand und Ereignisse
Server	app/ mit Server Components: Routing, initiale Daten und Zugriffsschutz
Server	app/actions/ und app/api/: Autorisierung, Eingabeprüfung und Anwendungsfälle
Beide Seiten	lib/: reine Typen, Gewichtung- und Bewertungsfunktionen ohne Datenbankzugriff
Nur Server	db/queries/ → db/index.ts → PostgreSQL: Abfragen und Persistenz

Tabelle 1: Schichten und Ausführungsorte der Next.js-Anwendung

app/dashboard/layout.tsx prüft die Sitzung und lädt Gesellschaften, Wohnungen und Ausschreibungen auf dem Server. Die Daten gehen an DashboardShell und DashboardProvider, die als Client Components Auswahl, Dialoge und lokalen Zustand verwalten. Die öffentliche Route app/bewerben/[listingId]/page.tsx lädt eine veröffentlichte Ausschreibung serverseitig; ApplicationForm verarbeitet die Eingabe im Browser. Das Formular ruft submitListingApplication als Server Action auf, wo die Eingaben erneut geprüft und gespeichert werden. Der CSV-Export ist ein separater Route Handler unter app/api/listings/[listingId]/applications/export/route.ts.

Die Funktionen in lib/answer-preferences.ts und lib/field-weights.ts sind bewusst unabhängig von React und PostgreSQL. Sie werden sowohl für die Vorschau und Rangliste im Dashboard als auch bei serverseitiger Validierung und beim Export genutzt. Clientseitige Berechnungen dienen der unmittelbaren Anzeige; Berechtigungen und verbindliche Schreiboperationen bleiben auf dem Server. Damit ist die Schichtung eine Trennung der Verantwortlichkeiten innerhalb eines Monolithen, keine vollständig isolierte Domain-Schicht.

5.3 Ebene 2: Persistenz

Das ORM bildet das relationale PostgreSQL-Schema auf TypeScript-Typen ab und verwaltet versionierte Migrationen. Nur serverseitiger Code erhält Zugriff auf die Datenbankverbin-

derung. Bewerbungsantworten und die zum Einreichungszeitpunkt verwendeten Formularfelder werden in `listing_applications` gespeichert; die Rangfolge wird aus diesen Daten berechnet.

`db/schema.ts` ist die verbindliche Drizzle-Definition für Benutzer, Sitzungen, Gesellschaften, Wohnungen, Ausschreibungen und Bewerbungen. SQL-Migrationen liegen versioniert in `drizzle/`. Der Pool und der Drizzle-Client befinden sich in `db/index.ts` und werden nicht in Client Components importiert.

6 Laufzeitsicht

6.1 Bewerbung absenden

1. Eine Bewerberin oder ein Bewerber öffnet die durch den Next.js App Router bereitgestellte Formularroute. Navigation, Footer und Formularrahmen werden serverseitig gerendert.
2. Interaktive Formulareile werden als Client Components aktiviert. Sie prüfen unmittelbar erkennbare Eingabefehler und senden die Bewerbung an einen serverseitigen Endpunkt der Next.js-Anwendung.
3. Der Endpunkt prüft Zugangsschlüssel, Ausschreibungsstatus und Eingaben erneut auf dem Server und ruft das Fachmodul auf.
4. Die Server Action speichert die Antworten und eine Kopie der verwendeten Formularfelder über Drizzle in PostgreSQL. Die Rangfolge wird beim Anzeigen der Bewerbungen aus den gespeicherten Daten berechnet.
5. Bei Erfolg zeigt die Client Component eine Bestätigung. Bei einem erwarteten Fehler bleiben die Eingaben erhalten und die Oberfläche zeigt eine verständliche Meldung.

6.2 Bewertete Bewerbungen laden

1. Eine angemeldete Mitarbeiterin oder ein angemeldeter Mitarbeiter ruft im Verwaltungsbereich eine Ausschreibung auf.
2. Das Dashboard-Layout prüft die Sitzung auf dem Server und lädt die eigenen Gesellschaften, Wohnungen und Ausschreibungen.
3. Für eine ausgewählte Ausschreibung ruft die Client Component eine Server Action auf. Diese prüft die Berechtigung und lädt die Bewerbungen aus PostgreSQL.
4. Die datenbankfreie Bewertungsfunktion berechnet im Browser die Rangfolge für die Anzeige. Der CSV-Export berechnet dieselbe Rangfolge separat auf dem Server.

6.3 Datenbank beim gemeinsamen Start noch nicht bereit

1. Docker Compose startet die Services db, migrate und web.
2. PostgreSQL initialisiert das Datenverzeichnis und meldet seinen Zustand über einen Healthcheck.

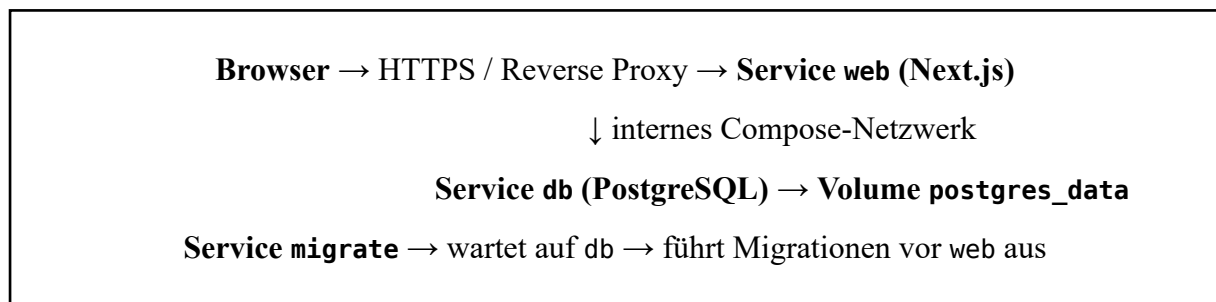
3. Nach erfolgreichem Healthcheck führt migrate die versionierten SQL-Migrationen aus. Erst nach dessen erfolgreichem Abschluss startet web. Schlägt ein Schritt fehl, startet der Web-Service nicht.

6.4 Aktueller API-Datenfluss

1. Ein Client ruft GET /api/applications auf.
2. Der Route Handler verwendet den serverseitigen Drizzle-Client und führt eine sortierte SELECT-Abfrage auf applications aus.
3. Bei Erfolg antwortet die Route mit { applications: [...] } und Cache-Control: no-store.
4. Bei einem unerwarteten Datenbankfehler wird der technische Fehler serverseitig protokolliert; der Client erhält eine allgemeine HTTP-500-Antwort ohne Verbindungsdetails.

7 Verteilungssicht

7.1 Container und Laufzeit



Code 1: Laufzeitdienste und Datenfluss der Compose-Umgebung

Knoten	Artefakt	Verantwortung
Browser	HTML und Client-JavaScript	Zeigt serverseitig erzeugte Seiten und führt interaktive Client Components aus.
Service web	Next.js-Standalone-Bundle aus dem Dockerfile	Server Components, Server Actions, Route Handlers, Authentisierung und Datenzugriff; intern auf Port 3000.
Service migrate	Eigenes Docker-Build-Target	Wendet mit <code>npm run db:migrate</code> versionierte Drizzle-Migrationen an und beendet sich danach.
Service db	PostgreSQL 17	Speichert relationale Daten; Healthcheck meldet die Bereitschaft. Der Port ist nur lokal an <code>127.0.0.1</code> gebunden.
Docker-Volume	postgres_data	Erhält Daten bei einer Neuerstellung der Container.

Das produktive Docker-Image wird mehrstufig gebaut. Nur das Standalone-Bundle, statische Dateien und öffentliche Assets gelangen in den laufenden Web-Container. Compose startet migrate nach erfolgreichem Datenbank-Healthcheck und web erst nach erfolgreicher Migration. So ist die Migrationsreihenfolge in der aktuellen Compose-Datei technisch festgelegt; fehlgeschlagene Migrationen verhindern den Start von web.

7.2 Reproduzierbarer Start

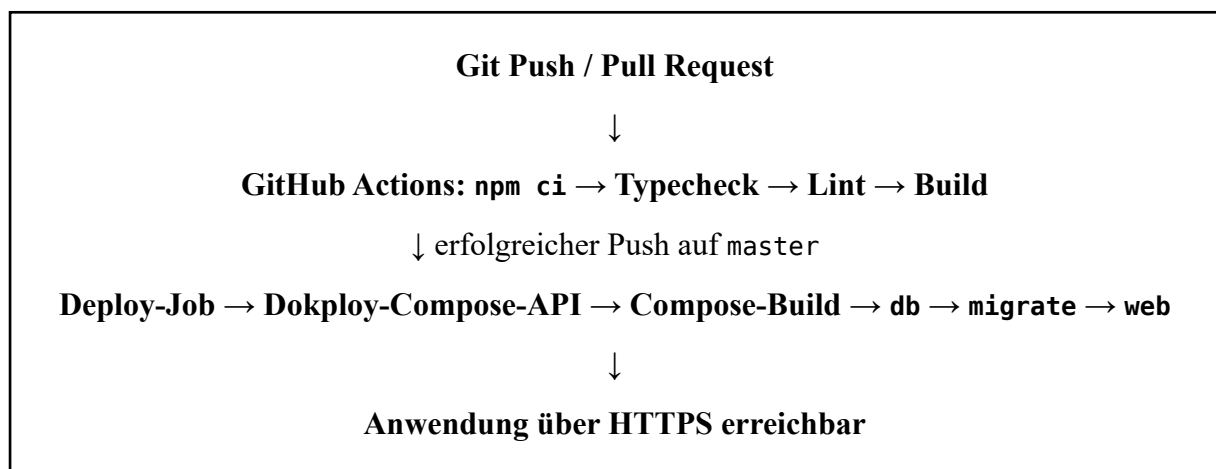
Voraussetzung ist Docker Compose. Eine `.env`-Datei beziehungsweise sicher gesetzte Umgebungsvariablen liefern Datenbankname, -benutzer, -passwort und gegebenenfalls erlaubte Origins für Server Actions. Echte Zugangsdaten werden nicht in Git eingchecked. Für den lokalen Browserzugriff ergänzt `docker-compose.local.yml` die Portfreigabe des Web-Services.

```
docker compose -f docker-compose.yml -f docker-compose.local.yml up --build
```

Die Anwendung ist lokal über Port 3000 erreichbar. `docker compose -f docker-compose.yml -f docker-compose.local.yml down` stoppt die Dienste und erhält das Datenbank-Volume. Schemaänderungen werden vorab mit `npm run db:generate` als SQL-Migration erzeugt und versioniert; der einmalige Service `migrate` wendet sie beim nächsten Compose-Start an.

7.3 CI/CD und automatisches Deployment mit Dokploy

Als Zielprozess soll jeder Push beziehungsweise Pull Request durch GitHub Actions geprüft werden. Nach einem erfolgreichen Push auf master soll ein nachgelagerter Deploy-Job die Dokploy-Compose-API aufrufen. Dokploy bezieht dann den neuen Commit und stellt den Compose-Stack bereit. API-Token und Compose-ID liegen als GitHub-Secrets vor; produktive Datenbankzugänge bleiben in Dokploy. Die direkte Auto-Deploy-Auslösung der GitHub-Integration muss für diesen Ablauf deaktiviert sein, damit kein ungeprüfter Push die CI-Stufen umgeht. Die [Dokploy-Dokumentation zu Auto Deploy](#) und die [Compose-API-Referenz](#) beschreiben die verfügbaren Auslöser.



Code 2: Vorgesehene CI/CD- und Deployment-Kette

Die Qualitätsstufen verwenden die bereits vorhandenen Skripte `npm run typecheck`, `npm run lint` und `npm run build`. Dokploy benötigt dazu das Git-Repository, den produktiven Branch, die Compose-Datei, eine Domain mit HTTPS sowie produktive Umgebungsvariablen. Vor dem Livebetrieb sind ausserdem Backup und Restore der PostgreSQL-Daten und ein überprüfbarer Umgang mit fehlgeschlagenen Deployments erforderlich.

Umsetzungsstand: Im Repository sind Dockerfile, Compose-Dateien und die Prüfbefehle vorhanden. Eine GitHub-Actions-Workflow-Datei ist derzeit nicht versioniert; eine Dokploy-Konfiguration ist im Repository nicht sichtbar. Die automatische Auslösung und ein erfolgreiches Live-Deployment sind daher mit dem vorliegenden Projektstand nicht nachgewiesen.

8 Querschnittliche Konzepte

8.1 Domänenmodell und Persistenz

Drizzle definiert in `db/schema.ts` Benutzer, Sitzungen, Gesellschaften, Wohnungen, Ausschreibungen und Bewerbungen. `listing_applications` speichert Antworten zusammen mit den zum Einreichungszeitpunkt verwendeten Formularfeldern. SQL-Migrationen werden versioniert und beim Compose-Start vor dem Web-Service ausgeführt.

8.2 API- und Schnittstellendesign

`GET /api/hello` und `GET /api/applications` liefern JSON. Der geschützte Route Handler für den Bewerbungs-Export liefert CSV. Fachliche Schreiboperationen laufen über Server Actions. Die Bewerbungsroute ist bewusst nicht zwischengespeichert und gibt bei unerwarteten Fehlern eine allgemeine HTTP-500-Antwort aus; technische Details bleiben im Server-Log.

8.3 Benutzeroberfläche und Responsive Design

Die Anwendung verwendet das Next.js-App-Layout mit deutscher Dokumentensprache und Metadaten. Dashboard, Dialoge und öffentliches Bewerbungsformular sind umgesetzt. Interaktive Bereiche sind Client Components; geschützte Daten werden serverseitig geladen und geprüft. Formale Accessibility-Tests sind noch offen.

8.4 Fehlerbehandlung und Logging

Konfigurationsfehler wie eine fehlende `DATABASE_URL` verhindern den Start des Datenbank-Clients. Datenbankfehler der API werden serverseitig mit Kontext geloggt, während der Client keine Zugangsdaten oder SQL-Details erhält.

8.5 Sicherheit

Zugangsdaten liegen in `.env` und sind von Git ausgeschlossen. Docker Compose verlangt ein explizites PostgreSQL-Passwort; der Datenbankport ist nur an `127.0.0.1` gebunden. Die Sitzung wird über ein HTTP-only-Cookie verwaltet. Das Dashboard prüft die Anmeldung, und serverseitige Aktionen und Exporte prüfen die Berechtigung für die jeweilige Ausschreibung. Eingaben werden serverseitig erneut validiert.

8.6 Testkonzept

Teststufe	Zweck	Beispiele/Werkzeuge
Statisch	Typen und Stilregeln prüfen	npm run typecheck und npm run lint
Build	Produktionsbundle und Route-Typen prüfen	npm run build
Integration	Datenbank und Schnittstellen prüfen	Manuelle Prüfung von GET /api/applications; automatisierte Tests noch offen
E2E/Qualität	Kritische Journeys, Performance und Accessibility prüfen	Noch offen; vor fachlicher Fertigstellung ergänzen

9 Architekturentscheidungen

Entscheidungen werden als kurze Architecture Decision Records (ADR) festgehalten. Eine Entscheidung beschreibt Kontext und Folgen, nicht nur das gewählte Werkzeug.

9.1 ADR-001: Next.js statt getrenntem React-Frontend und Express-Backend

Status	Akzeptiert am 2. September 2026
Kontext	Ursprünglich waren ein React-Frontend und ein separates Express-Backend vorgesehen. Für die Einzelarbeit sollen Routing, Typen, Betrieb und Deployment mit wenig Doppelkonfiguration beherrschbar bleiben. Serverseitige Validierung und geschützter Datenzugriff sind erforderlich.
Betrachtete Optionen	1. React-SPA mit Express-API; 2. Next.js als Full-Stack-Framework; 3. getrennte Frontend- und Backend-Anwendungen in einem Monorepo.
Entscheidung	Next.js mit App Router und TypeScript wird als modularer Monolith eingesetzt. Server Components rendern Navigation, Footer, Layouts und datenlesende Seiten. Client Components übernehmen interaktive Teilbereiche. Route Handlers und/oder Server Actions bilden serverseitige Endpunkte; ein Express-Service entfällt.
Begründung	Next.js bündelt Routing, Rendering und Backend-Funktionen. Gemeinsame Typen und ein Artefakt reduzieren Schnittstellen- und Betriebsaufwand. Serverorientiertes Rendering begrenzt das Browser-JavaScript und passt zum Projektumfang.
Konsequenzen	Weniger Konfiguration und kein separates API-Deployment stehen einer Bindung an Next.js-Konventionen gegenüber. Klare Modulgrenzen, kleine Client-Inseln, ausschliesslich serverseitiger Datenbankzugriff und automatisierte Tests verhindern eine Vermischung der Verantwortlichkeiten.

9.2 ADR-002: PostgreSQL und gemeinsamer Start mit Docker Compose

Status	Akzeptiert am 2. September 2026
Kontext	Ausschreibungen, dynamische Formulare, Kriterien, Gewichtungen, Bewerbungen und Bewertungen stehen in engen Beziehungen und müssen konsistent gespeichert werden. Die lokale Laufzeitumgebung soll auf verschiedenen Rechnern mit einem Befehl reproduzierbar sein.
Betrachtete Optionen	1. Eingebettete Datenbank; 2. lokal manuell installierte PostgreSQL-Datenbank; 3. PostgreSQL als Docker-Service zusammen mit der containerisierten Next.js-Anwendung.
Entscheidung	PostgreSQL wird als Service db betrieben. Die Next.js-Anwendung läuft als Service app. Docker Compose definiert Build, Netzwerk, Healthcheck, Umgebungsvariablen und ein persistentes Datenbank-Volume. Der gesamte Stack startet mit <code>docker compose up --build</code> .
Begründung	PostgreSQL unterstützt relationale Integrität und Transaktionen für das verknüpfte Fachmodell. Docker Compose vereinheitlicht Versionen und Startablauf, ohne eine lokal installierte Datenbank vorauszusetzen.
Konsequenzen	Docker ist für die lokale Ausführung erforderlich und benötigt zusätzliche Ressourcen. Datenbankmigrationen, Secrets sowie Backup und Restore bleiben explizite Betriebsaufgaben. Der Datenbankport wird standardmässig nicht nach aussen veröffentlicht; ein Volume verhindert Datenverlust bei einer Neuerstellung des Containers.

9.3 ADR-003: Drizzle ORM statt Prisma

Status	Akzeptiert am 18. September 2026
Kontext	Die Next.js-Anwendung benötigt einen typsicheren Zugriff auf PostgreSQL sowie versionierte Migrationen. Der technische Kern soll klein bleiben und SQL-Abfragen nachvollziehbar machen.
Betrachtete Optionen	1. Direkter Zugriff mit pg; 2. Prisma; 3. Drizzle ORM.
Entscheidung	Drizzle ORM wird mit dem node-postgres-Treiber eingesetzt. Tabellen werden in <code>db/schema.ts</code> definiert; Drizzle Kit erzeugt und wendet Migrationen im Verzeichnis <code>drizzle/</code> an.
Begründung	Drizzle verbindet aus dem TypeScript-Schema abgeleitete Typen mit SQL-nahen Abfragen. Gegenüber Prisma benötigt es keinen zusätzlichen Generierungsschritt für einen Client und hält die Abfragen für das kleine Projekt direkt lesbar. Gegenüber reinem pg reduziert es wiederholte Typdefinitionen und strukturiert Schema-Migrationen.
Konsequenzen	Datenbankzugriff bleibt auf Node.js-Routen beschränkt. Schemaänderungen benötigen weiterhin eine erzeugte und kontrolliert angewendete Migration. Teammitglieder müssen die Drizzle-Abfrage- und Migrationskonventionen kennen.

10 Qualitätsanforderungen

10.1 Qualitätsbaum

- Qualität
 - Nachvollziehbarkeit: deterministische Bewertung, erklärbare Teilbewertungen
 - Benutzbarkeit: responsive Bedienung, verständliche Rückmeldungen
 - Änderbarkeit: klare Verantwortlichkeiten, automatisierte Tests
 - Zuverlässigkeit: Datenintegrität, kontrollierte Fehlerbehandlung
 - Effizienz: Lighthouse-Durchschnitt mindestens 90 auf Mobile und Desktop

10.2 Qualitätsszenarien

ID	Merkmal	Szenario	Messgrösse	Nachweis
QS-01	Performance	Eine typische Seite wird in der Produktionsversion aufgerufen.	Lighthouse-Durchschnitt ≥ 90 , Mobile und Desktop	Lighthouse-Bericht
QS-02	Benutzbarkeit	Eine Kernfunktion wird auf einem kleinen Bildschirm ausgeführt.	Ohne horizontales Scrollen; alle Aktionen erreichbar	E2E/Manuell
QS-03	Zuverlässigkeit	Ungültige Daten werden übermittelt.	Keine inkonsistenten Daten; verständliche Fehlermeldung	Integration/E2E
QS-04	Änderbarkeit	Ein neues Bewertungskriterium oder ein neuer Formulartyp wird ergänzt.	Änderung bleibt auf klar abgegrenzte Bausteine beschränkt	Review/Tests
QS-05	Nachvollziehbarkeit	Eine gespeicherte Bewertung wird automatisch bewertet.	Gesamtwert ist aus Antworten, Kriterien und Gewichtungen reproduzierbar; alle Teilwerte werden ausgewiesen	Unit-/E2E-Test

10.3 Evaluationsergebnisse

[Ergebnisse, Abweichungen und getroffene Verbesserungsmaßnahmen]

11 Risiken und technische Schulden

Risiko/Schuld	Eintritt	Auswirkung	Gegenmassnahme	Status/Owner
Next.js-Server- und Clientgrenzen werden vermisch	Mittel	Hoch	Fachlogik in serverseitigen Modulen kapseln; Datenbankzugriff nie in Client Components; Architekturtests und Reviews	Offen / Noah Emmenegger
Docker-Start schlägt wegen nicht bereiter Datenbank fehl	Mittel	Mittel	PostgreSQL-Healthcheck; migrate wartet auf db, web auf die erfolgreiche Migration	Technisch abfangen / Noah Emmenegger
Geändertes Datenbankpasswort passt nicht zum persistierten Volume	Mittel	Mittel	Bei bestehendem Volume das ursprüngliche Passwort verwenden; für einen Neuaufbau bewusst docker compose down -v einsetzen	Dokumentiert / Noah Emmenegger
Migration schlägt beim Deployment fehl	Mittel	Hoch	Versionierte Migration im separaten Compose-Service migrate anwenden; web nur nach Erfolg starten	Technisch abfangen / Noah Emmenegger
Ungeprüfter Commit wird automatisch veröffentlicht	Mittel	Hoch	Dokploy-Deployment erst nach erfolgreicher CI auslösen und direkte GitHub-Auto-Deploy-Auslösung deaktivieren	CI/CD noch offen / Noah Emmenegger
[Lighthouse-Ziel wird spät verfehlt]	Mittel	Mittel	Regelmässige Messung mit Produktions-Build	[Offen / Name]
Backup und Restore noch nicht automatisiert	Mittel	Hoch	Vor einem Produktivbetrieb Backup- und Restore-Ablauf definieren und testen	Akzeptiert für lokale Entwicklung / Noah Emmenegger

12 Glossar

Begriff	Bedeutung im Projekt
App Router	Dateisystembasiertes Routing von Next.js für Seiten, Layouts und serverseitige Endpunkte
Client Component	React-Komponente, die für Browserzustand oder Interaktionen clientseitig ausgeführt wird
CRUD	Erstellen, Lesen, Ändern und Löschen einer Ressource
Docker Compose	Beschreibung und gemeinsamer Start der Container app und db samt Netzwerk, Konfiguration und Volume
Modularer Monolith	Eine gemeinsam ausgelieferte Anwendung, deren fachliche Verantwortlichkeiten intern in klar abgegrenzte Module gegliedert sind
MoSCoW	Priorisierung in Must, Should, Could und Won't
Route Handler	Serverseitiger HTTP-Endpunkt innerhalb des Next.js App Routers
Server Component	Standardmässig serverseitig gerenderte React-Komponente, zum Beispiel für Navigation, Footer oder datenlesende Seiten
Server Action	Serverseitig ausgeführte Next.js-Funktion, die von der Benutzeroberfläche aus aufgerufen werden kann

13 Fazit und Reflexion

13.1 Was ist gut gelaufen?

[Erfolgreiche Vorgehensweisen, technische Ergebnisse und Gründe]

13.2 Wo lagen die Herausforderungen?

[Fachliche, technische und organisatorische Herausforderungen sowie der Umgang damit]

13.3 Was würde ich beim nächsten Mal anders machen?

[Konkrete Verbesserungen für Planung, Architektur, Umsetzung und Qualitätssicherung]

13.4 Zielerreichung

[Erreichte und nicht erreichte User Stories sowie Qualitätsziele; Abweichungen begründen]

14 Arbeitsjournal

Datum	Stunden	Tätigkeiten/Ergebnis	Kumuliert
31.08.2026	3.0	Grundstruktur der arc42-Dokumentation und erstes Typst-Template angelegt.	3.0
01.09.2026	5.0	Dokumentationsvorlage integriert und angepasst; Kapitelstruktur, Titelseite und ersten Projektvorschlag ausgearbeitet. Architekten miteinander verglichen und evaluiert	8.0
02.09.2026	7.0	Projektvorschlag und Architekturkapitel zu Kontext, Lösung, Bausteinen und Qualität überarbeitet; PDFs erstellt.	15.0
16.09.2026	4.0	Umsetzung vorbereitet und Projektstruktur, Datenhaltung und lokale Entwicklungsumgebung geplant. Projektstruktur auswählen	19.0
17.09.2026	6.0	Next.js-Projekt aufgesetzt; Docker-Compose mit PostgreSQL, Drizzle ORM konfiguriert und integriert und Datenbankbindung eingerichtet.	25.0
18.09.2026	4.0	Beispiel-API mit Drizzle-Abfragen und erster Datenbankmigration umgesetzt; Ablauf im README beschrieben.	29.0
19.09.2026	4.0	API- und Docker-Konfiguration bereinigt; Architekturentscheidungen und technische Konzepte dokumentiert.	33.0
22.09.2026	5.0	Datenmodell und Abläufe für Benutzer, Firmen und Wohnungen vorbereitet; Dashboard-Struktur ausgearbeitet.	38.0
23.09.2026	9.0	Registrierung, Anmeldung, Firmenverwaltung und Dashboard umgesetzt; Wohnungsverwaltung, Ausschreibungsformular und Bewerbungsablauf ergänzt.	47.0

Datum	Stunden	Tätigkeiten/Ergebnis	Kumuliert
24.09.2026	8.0	Antwortpräferenzen und Feldgewichtung ergänzt; dynamischen Regler und Filter für die beste Übereinstimmung umgesetzt.	55.0
25.09.2026	8.0	Datenbankzugriff und Docker-Deployment korrigiert; Migration repariert sowie Bewerbungs-Export und Import einer Beispielwohnung ergänzt. Doku ergänzt und abgeschlossen	63.0
Total	63.0	Persönlicher Arbeitsaufwand	63.0

15 Anhang

15.1 Abgabe und Softwareartefakte

Artefakt	Fundort
Git-Repository	[URL]
Produktive Applikation	[URL oder Startanleitung]
Testberichte	[Pfad/URL]
Lighthouse-Berichte	[Pfad/URL]
API-Dokumentation	[Pfad/URL, falls vorhanden]

15.2 Ergänzende Unterlagen

[Zusätzliche, im Haupttext referenzierte Inhalte]